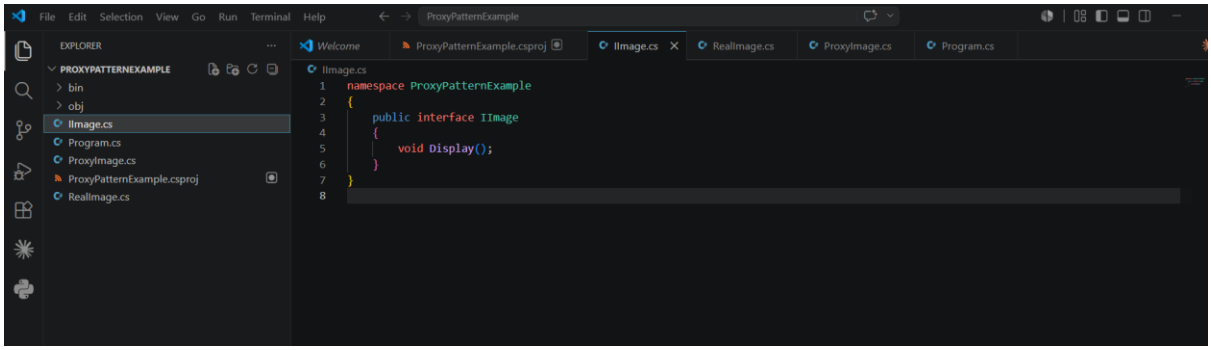
Hands-On Report

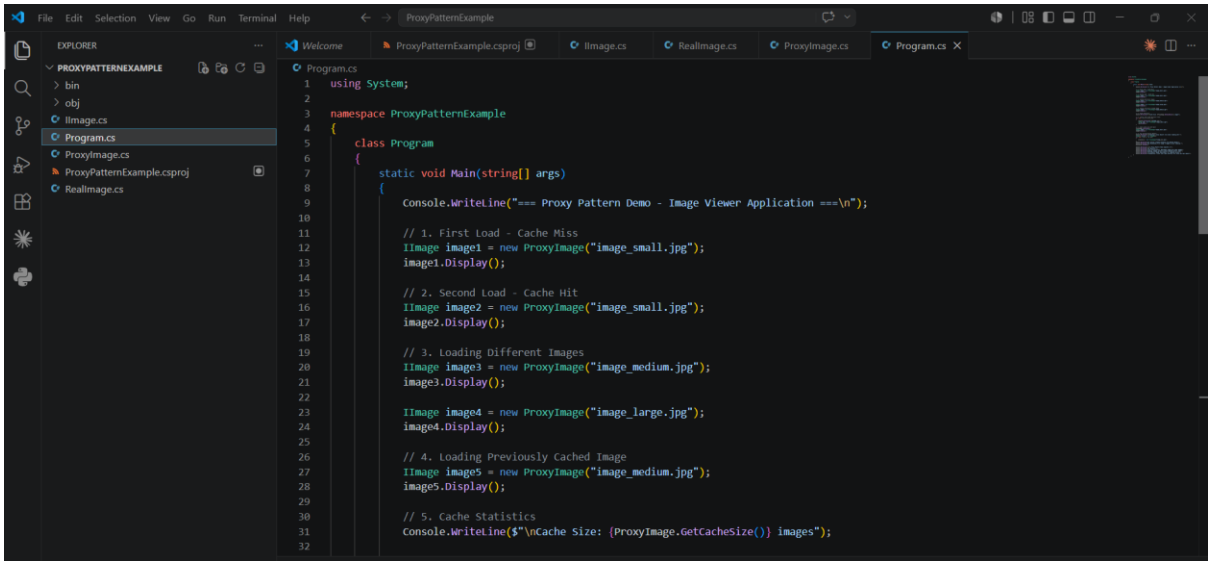
Exercise 6: Proxy Pattern Example

In this exercise, I implemented and tested the Proxy Pattern in a .NET application by developing a project named **ProxyPatternExample**. I created an **Image** interface and a **RealImage** class to represent images loaded from a remote source. A **ProxyImage** class was then developed to act as an intermediary, managing the creation of the real image only when it was needed.

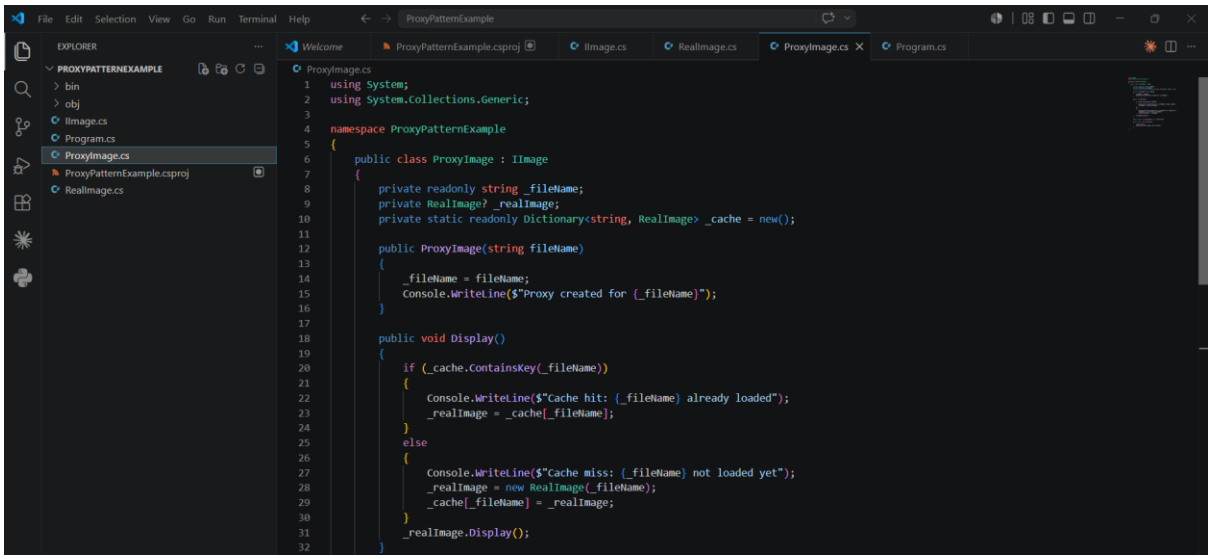
After executing the application, I verified that images were loaded only during their first access and reused on subsequent requests, confirming the effectiveness of lazy loading and caching. This practical exercise helped me understand how the Proxy Pattern improves application performance, minimizes unnecessary resource usage, and provides efficient, controlled access to resource-intensive objects in .NET applications.

Coding:

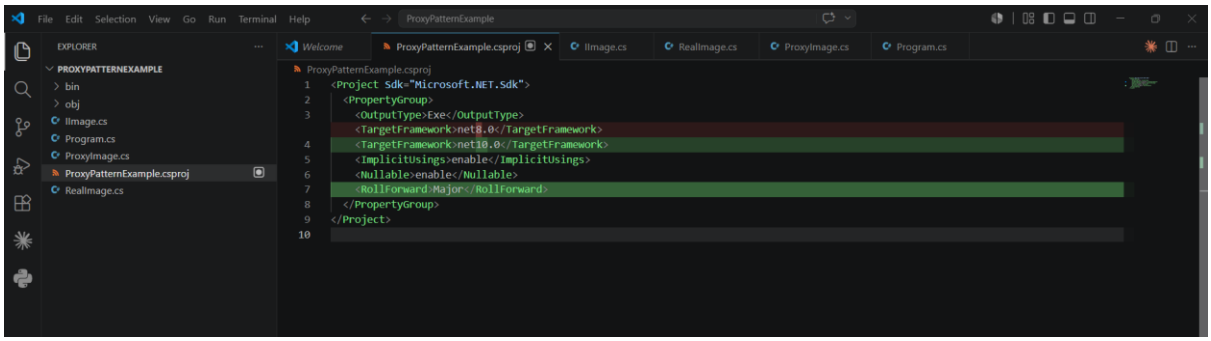




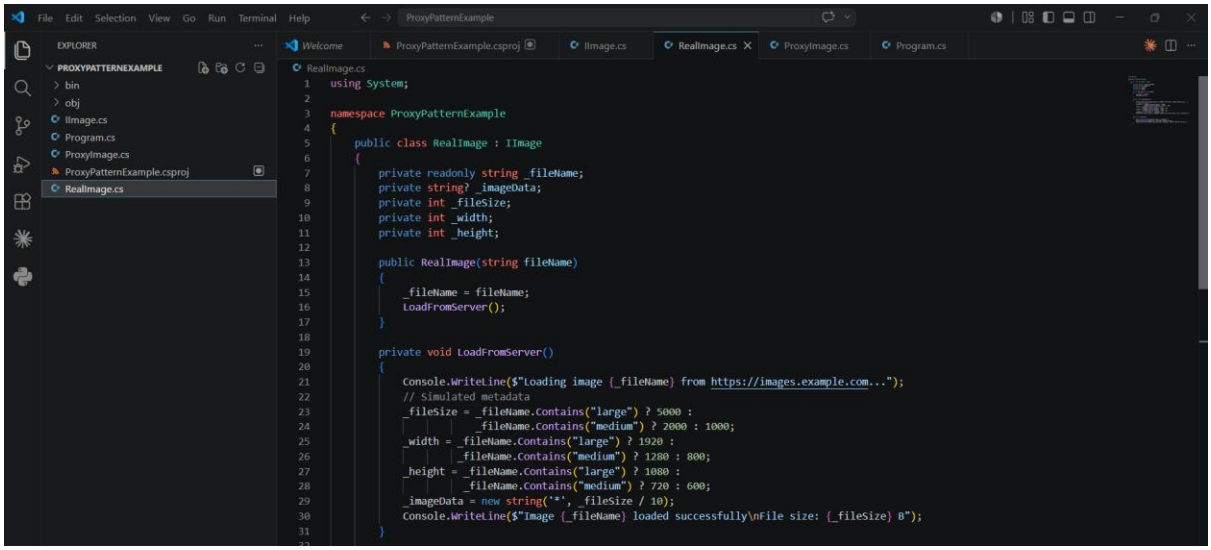
```
using System;

namespace ProxyPatternExample
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Proxy Pattern Demo - Image Viewer Application ===\n");

            // 1. First Load - Cache Miss
            IImage image1 = new ProxyImage("image_small.jpg");
            image1.Display();

            // 2. Second Load - Cache Hit
            IImage image2 = new ProxyImage("image_small.jpg");
            image2.Display();

            // 3. Loading Different Images
            IImage image3 = new ProxyImage("image_medium.jpg");
            image3.Display();

            IImage image4 = new ProxyImage("image_large.jpg");
            image4.Display();

            // 4. Loading Previously Cached Image
            IImage image5 = new ProxyImage("image_medium.jpg");
            image5.Display();

            // 5. Cache Statistics
            Console.WriteLine($"\nCache Size: {ProxyImage.GetCacheSize()} images");

```

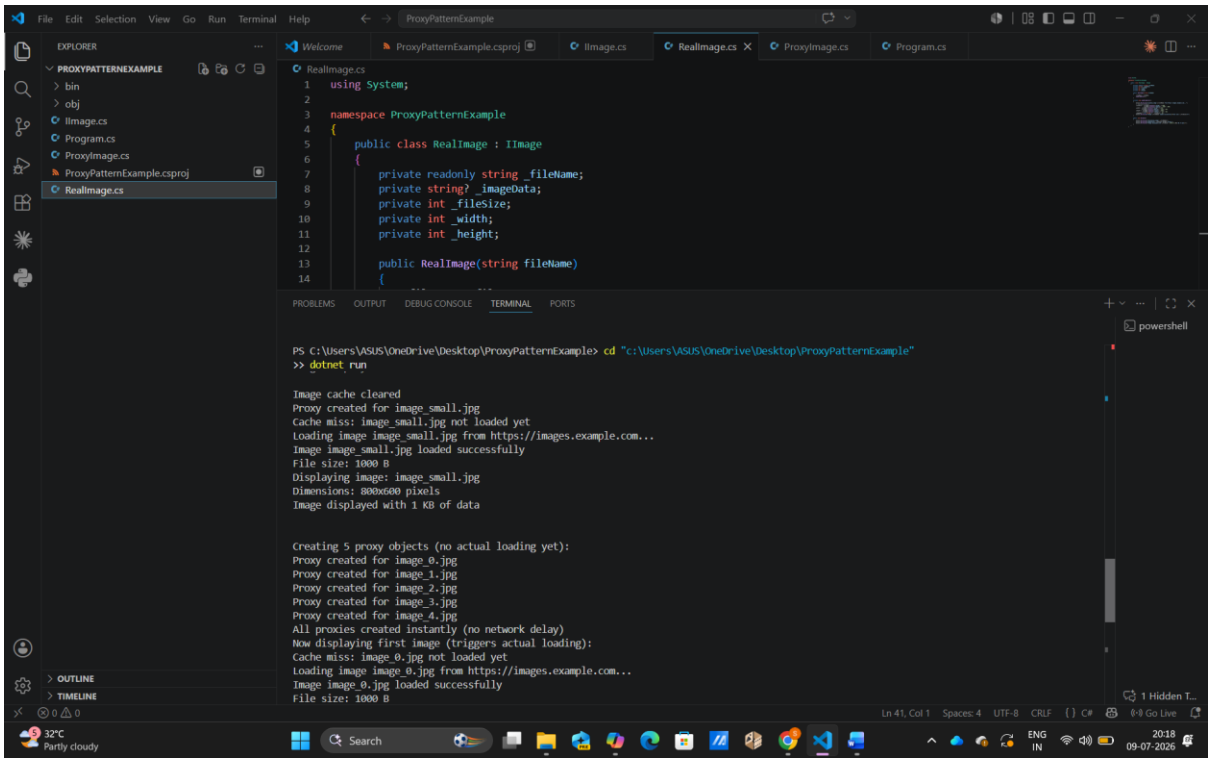


```
using System;
using System.Collections.Generic;

namespace ProxyPatternExample
{
    public class ProxyImage : IImage
    {
        private readonly string _fileName;
        private RealImage? _realImage;
        private static readonly Dictionary<string, RealImage> _cache = new();

        public ProxyImage(string fileName)
        {
            _fileName = fileName;
            Console.WriteLine($"Proxy created for {_fileName}");
        }

        public void Display()
        {
            if (_cache.ContainsKey(_fileName))
            {
                Console.WriteLine($"Cache hit: {_fileName} already loaded");
                _realImage = _cache[_fileName];
            }
            else
            {
                Console.WriteLine($"Cache miss: {_fileName} not loaded yet");
                _realImage = new RealImage(_fileName);
                _cache[_fileName] = _realImage;
            }
            _realImage.Display();
        }
```



```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <RollForward>Major</RollForward>
  </PropertyGroup>
</Project>
```


File Edit Selection View Go Run Terminal Help
ProxyPatternExample
EXPLORER
PROXYPATTERNEXAMPLE
bin
obj
IImage.cs
Program.cs
ProxyImage.cs
ProxyPatternExample.csproj
RealImage.cs
Welcome
ProxyPatternExample.csproj
IImage.cs
RealImage.cs
ProxyImage.cs
Program.cs
RealImage.cs
using System;
namespace ProxyPatternExample
{
public class RealImage : IImage
{
private readonly string _fileName;
private string? _imageData;
private int _fileSize;
private int _width;
private int _height;
public RealImage(string fileName)
{
_fileName = fileName;
LoadFromServer();
}
private void LoadFromServer()
{
Console.WriteLine($"Loading image {_fileName} from https://images.example.com...");
// Simulated metadata
_fileSize = _fileName.Contains("large") ? 5000 :
_fileName.Contains("medium") ? 2000 : 1000;
_width = _fileName.Contains("large") ? 1920 :
_fileName.Contains("medium") ? 1280 : 800;
_height = _fileName.Contains("large") ? 1080 :
_fileName.Contains("medium") ? 720 : 600;
_imageData = new string('*', _fileSize / 10);
Console.WriteLine($"Image {_fileName} loaded successfully\nFile size: {_fileSize} B");
}


File Edit Selection View Go Run Terminal Help
ProxyPatternExample
EXPLORER
PROXYPATTERNEXAMPLE
bin
obj
IImage.cs
Program.cs
ProxyImage.cs
ProxyPatternExample.csproj
RealImage.cs
Welcome
ProxyPatternExample.csproj
IImage.cs
RealImage.cs
ProxyImage.cs
Program.cs
RealImage.cs
using System;
namespace ProxyPatternExample
{
public class RealImage : IImage
{
private readonly string _fileName;
private string? _imageData;
private int _fileSize;
private int _width;
private int _height;
public RealImage(string fileName)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powershell
PS C:\Users\ASUS\OneDrive\Desktop\ProxyPatternExample> cd "c:\Users\ASUS\OneDrive\Desktop\ProxyPatternExample"
>> dotnet run
Image cache cleared
Proxy created for image_small.jpg
Cache miss: image_small.jpg not loaded yet
Loading image image_small.jpg from https://images.example.com...
Image image_small.jpg loaded successfully
File size: 1000 B
Displaying image: image_small.jpg
Dimensions: 800x600 pixels
Image displayed with 1 KB of data
Creating 5 proxy objects (no actual loading yet):
Proxy created for image_0.jpg
Proxy created for image_1.jpg
Proxy created for image_2.jpg
Proxy created for image_3.jpg
Proxy created for image_4.jpg
All proxies created instantly (no network delay)
Now displaying first image (triggers actual loading):
Cache miss: image_0.jpg not loaded yet
Loading image image_0.jpg from https://images.example.com...
Image image_0.jpg loaded successfully
File size: 1000 B
OUTLINE
TIMELINE
Ln 41, Col 1
Spaces: 4
UTF-8
CRLF
C#
Go Live
1 Hidden T...
32°C
Partly cloudy
Search
ENG
IN
20:18
09-07-2026